



نظري

6

23

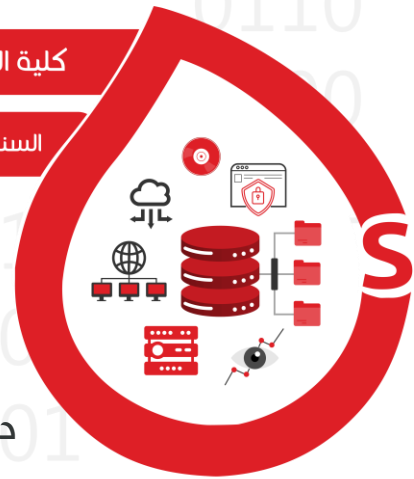
3450 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Query Processing - part2

د. مادلين عبود



RB Informatics; 23/11/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

Join Operations

عمليات الدمج

ما هي عملية ال join؟ ولماذا لا نستخدم Cartesian Product؟

عملية join هي عملية أساسية في معالجة الاستعلامات، ويمكن اعتبارها:

- تنفيذ ذكي لعملية (الضرب الديكارتي) Cartesian Product متبوعة ب Selection أي أنه نظرياً:

$$R \bowtie S \equiv \sigma (R \times S)$$

أي أنه عملياً في أنظمة قواعد البيانات:

- لا يتم تنفيذ Cartesian Product ثم Selection
- بل يتم تنفيذ خوارزميات Join مباشرة لأنها أقل كلفة بكثير

لأن:

- Cartesian Product يولد عدداً هائلاً من السجلات وكلفته عالية جداً
- بينما Join يستخدم خوارزميات تقلل عدد المقارنات وعمليات القراءة من القرص

لذلك:

- يستخدم ال DBMS عدة خوارزميات لل Join ويختار الأقل كلفة بناءً على تقدير الكلفة (Cost Estimation)

خوارزميات تنفيذ Join:

الخوارزميات الأساسية هي:

1. Nested-Loop Join الدمج بحلقة متداخلة على مستوى السطر
2. Block Nested-Loop Join الدمج بحلقة متداخلة على مستوى الكتلة
3. Indexed Nested-Loop Join الدمج بحلقة متداخلة باستخدام فهرس
4. Merge-Join الدمج الممزوج
5. Hash-Join الدمج باستخدام التقطيع

تقدير حجم ناتج عملية Join:

الحالة الأولى: لا توجد صفات مشتركة

- $R \cap S = \emptyset$
- الناتج هو Cartesian Product (عملية الدمج مطابقة للجداء الديكارتي)
- وعدد السجلات (الأسطر): $n_r \times n_s$

الحالة الثانية: Natural Join على صفة A (دمج السجلات من العلاقتين فقط عندما تكون قيمة الصفة A متساوية فيهما)

- $R \cap S = \{A\}$
- عدد السجلات الناتجة يقدر ب: $n_r \times n_s / V(A, r)$ أو $n_r \times n_s / V(A, s)$

حيث:

- $V(A, r)$ عدد القيم المختلفة للصفة A في العلاقة r
- $V(A, s)$ عدد القيم المختلفة للصفة A في العلاقة s

ملاحظة:

كلفة ال Join تتناسب طردياً مع عدد السجلات الناتجة.

Nested-Loop Join على مستوى السطر

الفكرة:

لدينا حلقتان متداخلتان:

- لكل سجل في العلاقة الأولى r
- نمر على جميع سجلات العلاقة الثانية s
- نختبر شرط الدمج

الخوارزمية لتنفيذ عملية دمج وفق θ : أي $R \bowtie_{\theta} S$ (دمج كل سجل من العلاقة R مع كل سجل من العلاقة s ثم نحتفظ فقط بالأزواج التي تحقق شرط الدمج θ)

```
for each tuple tr in r do begin
  for each tuple ts in s do begin
    test pair (tr, ts) to see if they satisfy the join condition  $\theta$ 
  if they do, add tr • ts to the result
  end
end
```

العلاقة الخارجية = r Outer Relation

العلاقة الداخلية = s Inner Relation

سجل من العلاقة r = tr

سجل من العلاقة s = ts

سجل جديد يحتوي أعمدة r و s = $tr \cdot ts$

الخصائص:

1. لا يحتاج إلى فهرس
2. يعمل مع أي شرط دمج
3. يفحص كل زوج ممكن من السجلات
4. كلفته عالية جداً

⚡ كلفة Nested-Loop Join (أسوأ حالة):

إذا كانت الذاكرة تتسع لكتلة واحدة فقط من كل علاقة (أسوأ حالة من توفر الذاكرة):

$$\text{Cost} = nr \times bs + br$$

حيث:

nr عدد سجلات العلاقة الخارجية

bs عدد كتل العلاقة الداخلية

br عدد كتل العلاقة الخارجية

⚡ لماذا هذه الصيغة؟

- لكل سجل واحد من العلاقة الخارجية:
- نقرأ كل كتل العلاقة الداخلية
- وبعد الانتهاء:
- نقرأ كتل العلاقة الخارجية مرة واحدة

لذلك عند استخدام هذا النوع من الدمج القاعدة الذهبية هي جعل العلاقة الأصغر هي الـ Inner Relation لأنها تقرأ عدداً كبيراً من المرات

مثال رقمي:

depositor

• 5000 سجل

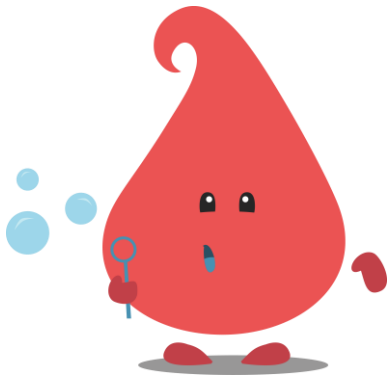
• 100 كتلة

• سعة الكتلة = $5000/100 = 50$ تسجيلة

customer

• 10000 سجل

• 400 كتلة



• | ساعة الكتلة = $10000/400 = 25$ تسجيلة

ملاحظة:

أسوأ حالة من توفر الذاكرة تعني الذاكرة المتاحة صغيرة جداً، ولا تسمح إلا بتحميل كتلة واحدة فقط من كل علاقة في كل مرة.

الحالة 1: Outer depositor هي

$$5000 \times 400 + 100 = 2,000,100 \text{ disk accesses}$$

الحالة 2: Outer customer هي

$$10000 \times 100 + 400 = 1,000,400 \text{ disk accesses}$$

← **الكلفة أقل عندما تكون العلاقة الأصغر هي ال Inner**

الحالة خاصة: إذا كانت depositor بالكامل في الذاكرة:

ماذا يعني هذا؟

- نحمل كل depositor (100 كتلة) مرة واحدة إلى الذاكرة
- لا نعيد قراءته من القرص

الكلفة تصبح:

- قراءة depositor مرة واحدة = 100
- قراءة customer مرة واحدة = 400

$$\text{Cost} \approx 500 \text{ disk accesses}$$

**الخلاصة:**

إذا كانت العلاقة الداخلية صغيرة ويمكن تحميلها في الذاكرة:

$$\text{الكلفة} \approx \text{مجموع كتل العلاقتين} (bs + br)$$

Block Nested-Loop Join على مستوى الكتل

الفكرة:

بدل المقارنة سطر بسطر نقارن كتلة بكتلة.

الخوارزمية:

```

for each block  $B_r$  of  $r$  do
begin
  for each block  $B_s$  of  $s$  do
  begin
    for each tuple  $t_r$  in  $B_r$  do
    begin
      for each tuple  $t_s$  in  $B_s$  do
      begin
        Check if  $(t_r, t_s)$  satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result.
      end
    end
  end
end
end

```

الفكرة الأساسية: نقارن كل كتلة من العلاقة الخارجية مع كل كتلة من العلاقة الداخلية، ونقارن السجلات داخل الكتل في الذاكرة.

يعني:

5. نحمل كتلة واحدة من r
6. نحمل كتلة واحدة من s
7. نقارن كل السجلات داخل هاتين الكتلتين
8. ننتقل للكتلة التالية

بالنسبة للكلفة:

الحالة الأولى: أسوأ حالة (ذاكرة صغيرة)

الذاكرة لا تكفي إلا لكتلة واحدة من كل علاقة

ماذا يحدث؟

- لكل كتلة من العلاقة الخارجية r
- نقرأ كل كتل العلاقة الداخلية s

الكلفة:

▪ $Cost = br \times bs + br$

لماذا؟

- $br \times bs$: لكل كتلة خارجية نمر على كل كتل الداخلية
- $+ br$: قراءة العلاقة الخارجية نفسها

لاحظ:

- هذه أقل بكثير من: $nr \times bs + br$
- لأننا نعمل على مستوى الكتل وليس السجلات

الحالة الثانية: أفضل حالة (ذاكرة كافية)

الذاكرة تكفي لتخزين العلاقتين كاملتين

ماذا يحدث؟

- نقرأ كل علاقة مرة واحدة فقط.

الكلفة:

▪ $Cost = br + bs$

← هذه أفضل حالة ممكنة

النجاح لا يحتاج صخباً يحتاج التزاماً طويل النفس

تحسينات على خوارزمية Block Nested-Loop Join

إذا كان حجم الذاكرة المتاحة لتنفيذ العملية هو M كتلة، فيمكن استخدام:

- $M - 2$ كتلة لشحن معطيات العلاقة الخارجية
- كتلة واحدة لشحن العلاقة الداخلية
- وكتلة واحدة لتخزين الناتج (output)

وبالتالي يتم تقسيم br على $M-2$ ، أي بدل أن نأخذ في كل مرة كتلة واحدة فقط من العلاقة الخارجية، نأخذ $M-2$ كتلة دفعة واحدة.

- فتصبح الكلفة: $\lceil br / (M-2) \rceil \times bs + br$
- أي يتم تخفيض الكلفة بشكل أكبر.

توضيح فكرة استخدام $M-2$ من الذاكرة:

ما هو M أصلاً؟

- $M =$ عدد الكتل (Buffers) المتاحة في الذاكرة الرئيسية (RAM) أثناء تنفيذ الدمج.

مثال:

إذا كانت الذاكرة تستطيع حمل 10 كتل: $M = 10$

لماذا لا نستخدم كل M ؟

في Block Nested-Loop Join تحتاج الذاكرة لثلاثة أشياء ثابتة:

1. كتلة واحدة لقراءة كتل العلاقة الداخلية s
2. كتلة واحدة لكتابة النتائج (output buffer)
3. باقي الكتل نستخدمها لتحميل كتل من العلاقة الخارجية r

المقصود بتحميل كتل من العلاقة الخارجية هو جلب عدة كتل كاملة (Blocks / Pages) من العلاقة الخارجية r من القرص إلى الذاكرة في آن واحد، وليس تحميل سجل واحد أو العلاقة كاملة. بما أن العلاقة r مخزنة على القرص على شكل br كتلة، فإن استخدام $(M-2)$ كتلة من الذاكرة يسمح بتحميل دفعة واحدة من كتل العلاقة الخارجية في كل مرة، ثم تتم مقارنة جميع سجلات هذه الكتل مع جميع كتل العلاقة الداخلية s دون الحاجة لإعادة قراءة العلاقة الداخلية لكل كتلة منفردة من r . وبذلك ينخفض عدد مرات المرور على العلاقة الداخلية من br مرة إلى $\lceil br / (M-2) \rceil$ مرة فقط، مما يؤدي إلى تقليل عدد عمليات الوصول إلى القرص (disk accesses) وبالتالي تخفيض الكلفة الكلية لعملية الدمج.

كل عادة صغيرة اليوم

تصبح مصيراً كبيراً غداً



ملاحظات نهائية:

1. في حال كانت واصفة الدمج (join key) مفتاحاً في العلاقة الداخلية، تتوقف الحلقة الداخلية مباشرة عند الوصول إلى أول تطابق (لأننا نكون متأكدين أنه لا يوجد تطابق آخر لأن المفتاح فريد)
2. يمكن مسح العلاقة الداخلية باتجاه واحد أو من الاتجاهين للاستفادة من الكتل الموجودة في الذاكرة: المقصود بالمشح (Scan): طريقة قراءة كتل العلاقة الداخلية من القرص
 - اتجاه واحد (One-way scan): نبدأ من أول كتلة ونكمل حتى الأخيرة في كل مرة.
 - اتجاهين (Two-way / Bi-directional scan): نستفيد من الكتل التي ما زالت موجودة في الذاكرة (buffer) دون إعادة قراءتها من القرص.

الفكرة الأساسية:

إذا كانت بعض كتل العلاقة الداخلية ما زالت في الذاكرة من دورة سابقة:

- لا نعيد قراءتها من القرص
- نستفيد منها مباشرة

النتيجة:

- تقليل عدد block accesses
3. في حال وجود فهارس، يمكن الاستفادة منها لتقليل الكلفة أكثر (تحويل الخوارزمية عملياً إلى Indexed Nested-Loop Join)

الدمج بحلقات متداخلة باستخدام الفهارس Indexed Nested-Loop Join

متى نستخدم Indexed Nested-Loop Join؟

لا يمكن تطبيق هذه الخوارزمية إلا إذا تحقّق أحد الشروط التالية:

1. الدمج هو دمج طبيعي (Natural Join) أو دمج مساواة (Equi-Join)
أي أن شرط الدمج هو تساوي قيم واصفة الدمج.
2. وجود فهرس (Index) على واصفة الدمج في العلاقة الداخلية
(ويمكن حتى إنشاء فهرس خصيصاً فقط لتنفيذ عملية الدمج)

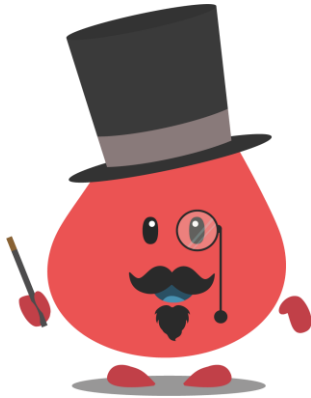
فكرة الخوارزمية:

بدل أن نقوم بمسح كامل العلاقة الداخلية لكل سجل من العلاقة الخارجية، نقوم باستخدام الفهرس للوصول مباشرة إلى السجلات المطابقة

الخوارزمية تكون بالشكل التالي:

For each tuple tr in the outer relation r :
Use the index to look up tuples in s
that satisfy the join condition with tuple tr

أي:



- نأخذ سجلاً واحداً tr من العلاقة الخارجية r
- نستخدم الفهرس للبحث في العلاقة الداخلية s
- نُخرج السجلات المتطابقة فقط
- من أجل أسوأ حالة من حيث الذاكرة:

أي نفترض:

- الذاكرة تكتفي كتلة واحدة فقط من العلاقة الخارجية r
- لكل سجل من r نقوم بعملية بحث عبر الفهرس في s

تكون كلفة Indexed Nested-Loop Join المتوقعة:

$$\text{Cost} = br + nr \times c$$

حيث:

- br : عدد كتل العلاقة الخارجية
- nr : عدد السجلات (الأسطر) في العلاقة الخارجية
- c : كلفة البحث في الفهرس + جلب السجلات المطابقة من العلاقة الداخلية
- عملياً c هو تقريباً ارتفاع شجرة الفهرس + 1
- (1+) للوصول إلى البيانات الفعلية بعد الوصول إلى ورقة الفهرس

ملاحظة مهمة:

إذا توفرت فهرس على واصفة الدمج في كلتا العلاقتين r و s فمن الأفضل اختيار العلاقة الأصغر بعدد الأسطر لتكون العلاقة الخارجية وذلك لتقليل عدد مرات البحث في الفهرس.

مثال توضيحي:

المعطيات:

العلاقة customer:

- 10000 سجل
- 25 سجل في الكتلة ← 400 كتلة
- يوجد فهرس Primary B+-Tree على customer-name
- 20 مدخلاً في كل عقدة فهرس
- ارتفاع الشجرة = 4
- نحتاج 1+ للوصول للبيانات ← $c = 5$

العلاقة depositor:

- 5000 سجل
- 50 سجل في الكتلة ← 100 كتلة
- depositor كعلاقة خارجية

الكلفة بدون استخدام الفهرس:(Block Nested-Loop Join): $br \times bs + br = 100 \times 400 + 100 = 40,100$ disk accessesالكلفة باستخدام الفهرس:(Indexed Nested-Loop Join): $br + nr \times c = 100 + 5000 \times (4 + 1) = 25,100$ disk accessesالاستنتاج:

- استخدام الفهرس يقلل الكلفة بشكل واضح.

Merge-Join (Sort-Merge Join) الدمج الممزوج

متى نستخدم Merge-Join؟

نستخدم فقط في الحالات التالية:

1. الدمج هو دمج مساواة (Equi-Join) أو دمج طبيعي (Natural Join)
 2. توجد واصفة دمج مشتركة بين العلاقتين
 3. العلاقتان مفروztان حسب واصفة الدمج
- وإن لم تكونا مفروztين نقوم بفرزهما أولاً

عملية الدمج مشابهة لمرحلة الدمج في خوارزمية الفرز بالدمج، الخلف بالتعامل مع القيم المكررة لواصفة الدمج حيث يتم احتساب كل زوج له نفس قيمة واصفة الدمج في النتيجة.

الخوارزمية (Conceptually):

```

(1) if R1 and R2 not sorted, sort them
(2) i ← 1; j ← 1;
    While (i ≤ T(R1)) ∧ (j ≤ T(R2)) do
        if R1[i].C = R2[j].C then OutputTuples
        else if R1[i].C > R2[j].C then j ← j+1
        else if R1[i].C < R2[j].C then i ← i+1
  
```

شرح الخوارزمية:

- في حال كانت الجداول غير مرتبة بحسب واصفة الدمج، نقوم أولاً بترتيبها.
 أما إذا كانت مرتبة مسبقاً، فنبدأ بتنفيذ الخوارزمية مباشرة.
- نستخدم مؤشرين، أحدهما لكل جدول، وندخل في حلقة while تستمر طالما أن المؤشرين ضمن حدود الجدولين.
 في كل خطوة نقارن قيمة واصفة الدمج في الجدول الأول مع قيمتها في الجدول الثاني:
- إذا كانت القيمتان متساويتين، نقوم باستدعاء التابع OutputTuples لإخراج جميع الأزواج المتطابقة.
 - إذا كانت قيمة الجدول الأول أكبر، نزيد مؤشر الجدول الثاني: $R1[i].C > R2[j].C$ نحرك j
 - وإذا كانت قيمة الجدول الأول أصغر، نزيد مؤشر الجدول الأول: $R1[i].C < R2[j].C$ نحرك i

تابع Output-Tuples (للقيم المكررة):

```

While (R1[ i ].C = R2[ j ].C) ∧ (i ≤ T(R1)) do
  {j ← j;
  while (R1[ i ].C = R2[ j ].C) ∧ (j ≤ T(R2)) do
    {
      output pair R1[ i ], R2[ j ];
      j ← j+1
    }
  i ← i+1 }
  
```

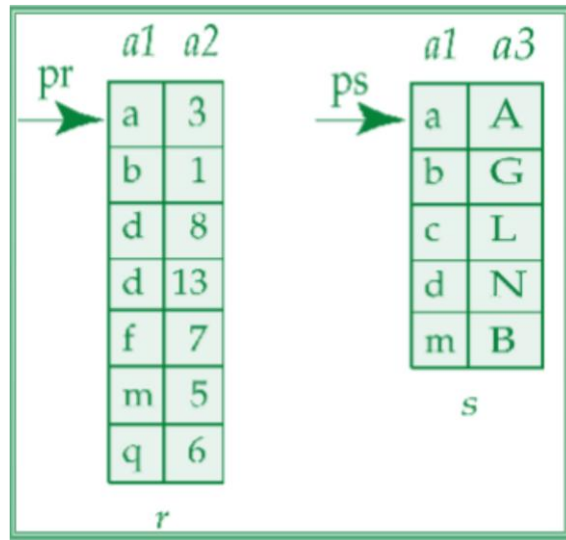
عمل التابع Output Tuples:

- طالما أن قيم واصفة الدمج متساوية في كلا الجدولين وما زلنا ضمن حدود الجدول الأول، نقوم بتثبيت موضع المؤشر في الجدول الثاني، ثم ندخل في حلقة داخلية تتحقق من استمرار التساوي مع بقية القيم في الجدول الثاني.
- في حال التساوي، يتم إخراج الزوج الناتج إلى الجدول النهائي، ثم نزيد مؤشر الجدول الثاني.
- بعد انتهاء الحلقة الداخلية، نزيد مؤشر الجدول الأول ونكرر العملية لإخراج جميع التركيبات الممكنة للقيم المتساوية.
- وبما أن كل كتلة من كل علاقة تقرأ مرة واحدة فقط أثناء مرحلة الدمج، فإن كلفة الخوارزمية تساوي: $br + bs$ كلفة الترتيب في حال كانت الجداول غير مرتبة +

ما تعرب منه اليوم**ستواجه غداً بثمن أعلى**

مثال تطبيقي:

ليكن لدينا الجدولين r و s كالتالي:



تم تطبيق خوارزمية Merge-Join على العلاقتين كما يلي:

- نضع مؤشرين في بداية الجدولين s و r ، ثم نقارن أول تسجيلية في الجدول r مع أول تسجيلية في الجدول s نلاحظ تطابق قيمة واصفة الدمج عند المفتاح a في كلا الجدولين، لذلك نقوم بدمج السجلين وإخراج الناتج $(a, 3, A)$.
- بعد ذلك نحرك المؤشر ps في الجدول s إلى التسجيلية التالية، فنجد أن قيمتها لا تتطابق مع التسجيلية الحالية في الجدول r ، لذا نحرك المؤشر pr في الجدول r إلى التسجيلية التالية، فنجد تطابقاً عند المفتاح b ، وبالتالي ندمج السجلين ونخرج $(b, 1, G)$
- نقوم بعدها بتحريك المؤشر ps في الجدول s ، فنلاحظ عدم تطابق القيمة مع المفتاح b ، لذلك نحرك المؤشر pr في الجدول r ، ولكن يبقى عدم التطابق قائماً
- نستمر بتحريك المؤشر المناسب (حسب مقارنة القيم) إلى أن نصل إلى المفتاح d في كلا الجدولين، حيث يتحقق شرط الدمج، فنخرج $(d, 8, N)$.
- نحرك المؤشر ps مرة أخرى، فنلاحظ عدم تطابق المفتاح التالي مع d ، لذلك نحرك المؤشر pr إلى التسجيلية التالية في الجدول r والتي تحمل أيضاً المفتاح d ، فنقوم بدمجها مع نفس القيمة في الجدول s ونخرج $(d, 13, N)$
- نستمر بهذه الآلية حتى انتهاء أحد الجدولين

وبذلك يكون الجدول الناتج هو ناتج دمج العلاقتين باستخدام خوارزمية Merge-Join:

a	3	A
b	1	C
d	8	N
d	13	N
m	5	B

- يمكن أيضًا تحسين خوارزمية Merge-Join بدمجها مع خوارزمية الدمج باستخدام الفهرس، لنحصل على خوارزمية أكثر كفاءة تسمى Hybrid Merge-Join.
 - في هذه الحالة تكون إحدى العلاقتين مفروزة حسب واصفة الدمج، بينما تكون العلاقة الأخرى غير مفروزة ولكن لديها فهرس ثانوي من نوع B+-Tree على واصفة الدمج.
- تنفذ الخوارزمية كما يلي:

- يتم دمج العلاقة المفروزة مباشرةً مع أوراق فهرس الـ B+-Tree الخاصة بالعلاقة غير المفروزة، بدلاً من مسح العلاقة غير المفروزة كاملة.
- ينتج عن ذلك ناتج وسيط يحتوي على عناوين الأسطر (Record Addresses) في العلاقة غير المفروزة
- بعد ذلك تُفرز هذه العناوين حسب ترتيب التخزين الفيزيائي (يعني ترتيب مؤشرات السجلات بحيث يتم الوصول إلى الكتل على القرص بشكل تسلسلي)
- أخيراً يتم مسح العلاقة غير المفروزة تسلسلياً وفق ترتيبها الفيزيائي، ودمجها مع الناتج الوسيط لاستبدال العناوين بالأسطر الفعلية.

بهذه الطريقة نستفيد من القراءة التسلسلية بدل الوصول العشوائي، مما يؤدي إلى تقليل الكلفة وتحسين الأداء مقارنة باستخدام الفهرس أو الدمج التقليدي كل على حدة.

Hash-Join (الدمج باستخدام التقطيع)

متى نستخدم Hash-Join؟

تستخدم خوارزمية Hash-Join فقط في حال:

- Natural Join
- Equi-Join (شرط الدمج هو مساواة)

في هذا النوع من الدمج نستخدم دالة هاش h لتقسيم أسطر كل من العلاقتين إلى مجموعات (أجزاء) بحيث أن الأسطر التي لها نفس قيمة واصفة الدمج تذهب إلى نفس الجزء.

مثال سريع:

إذا كانت دالة الهاش $h(x) = x \bmod 3$

وكانت قيمة واصفة الدمج = 5

فإن $h(5) = 2$

أي كل سطر قيمته 5 في r و s يوضع في الجزء رقم 2

وبالتالي:

- أي سطرين يمكن أن يندمجا سيكونان في نفس الجزء



لا تقس نفسك بالآخرين قسها بما كنت عليه أمس

توضيح مفصل:

كيف تعمل الدالة h :1. h maps JoinAttrs values to $\{0, 1, \dots, n\}$

- دالة الهاش h تأخذ قيمة واصفة الدمج (Join Attribute) وتحولها إلى رقم بين 0 و n
- هذا الرقم هو رقم الجزء (partition) الذي سياتي إليه السطر

2. JoinAttrs denotes the common attributes of r and s used in the natural join

- JoinAttrs هي الواصفة المشتركة بين العلاقتين r و s
- وهي الواصفة التي يتم الدمج الطبيعي بناءً عليها

مثلاً: إذا كان

 $r(\text{account_no}, \text{name})$ $s(\text{account_no}, \text{balance})$

فإن:

JoinAttrs = account_no

3. $r_0, r_1, \dots, r_n$ denote partitions of r tuples

- بعد تطبيق دالة الهاش على العلاقة r
- نقسمها إلى أجزاء: $0, r_1, r_2, \dots, r_n$
- كل جزء يحتوي مجموعة من أسطر r

4. Each tuple $tr \in r$ is put in partition r_i where $i = h(tr[\text{JoinAttrs}])$

أي:

- كل سطر tr في العلاقة r
- نأخذ قيمة واصفة الدمج فيه
- نطبق عليها دالة الهاش
- نضع السطر في الجزء r_i

مثلاً:

 $tr.\text{account_no} = 25$ $h(25) = 1$ بالتالي tr يوضع في r_1

5. $s_0, s_1, \dots, s_n$ denote partitions of s tuples

▪ نقسم العلاقة s بنفس الطريقة، إلى الأجزاء:

$s_0, s_1, s_2, \dots, s_n$

6. Each tuple $ts \in s$ is put in partition s_i where $i = h(ts[\text{JoinAttrs}])$

▪ كل سطر ts في العلاقة s ، نطبق نفس دالة الهاش h ، ونضعه في الجزء s_i

مثلاً:

$ts.\text{account_no} = 25$

$h(25) = 1$

بالتالي ts يوضع في s_1 .

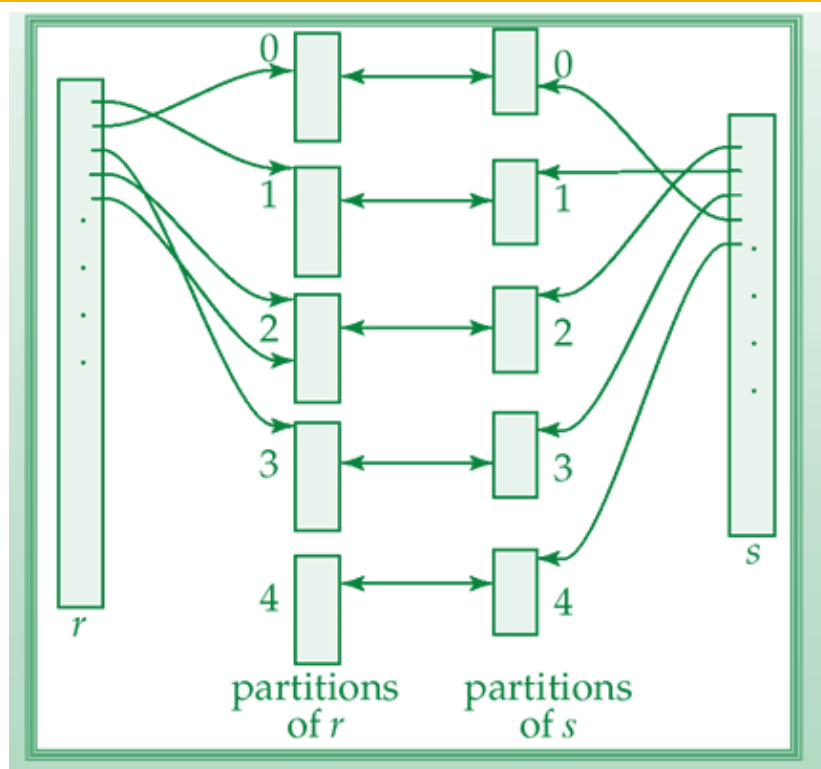
في النهاية أي سطرين قابلين للدمج سيكونان دائماً في نفس الجزء، وبالتالي نحتاج فقط لمقارنة r_i مع s_i .

ملاحظة:

Hr_i هو Partition يمثل مجموعة من الأسطر (tuples) من العلاقة r ، التي أعطت نفس ناتج الدالة h

Hs_i هو Partition يمثل مجموعة من الأسطر (tuples) من العلاقة s ، التي أعطت نفس ناتج الدالة h

بالتالي لإيجاد ناتج الدمج، نقارن فقط الأسطر الموجودة في الجزء Hr_i من العلاقة r مع الأسطر الموجودة في الجزء المقابل Hs_i من العلاقة s ، حيث كلا الجزأين ناتجان عن نفس قيمة الدالة h على واصفة الدمج.



خطوات عملية الدمج باستخدام التقطيع للعلاقين r و s كما يلي:

1. تجزئة العلاقة s باستخدام دالة التجزئة h وأثناء هذه العملية:

- تحجز كتلة ذاكرة واحدة كـ $input\ buffer$.
- وتُحجز كتلة $output\ buffer$ لكل جزء ناتج عن التجزئة.

2. تجزئة العلاقة r بنفس الطريقة تماماً

3. لكل جزء i على حدة نقوم بما يلي:

1. مرحلة البناء (Build):

- تحميل الجزء Hsi من العلاقة s كاملاً إلى الذاكرة.
- بناء فهرس Hash داخلي في الذاكرة على واصفة الدمج.
- نستخدم هنا دالة Hash مختلفة عن الدالة h المستخدمة في التجزئة.

تسمى العلاقة s في هذه المرحلة $Build\ Input$

2. مرحلة الفحص (Probe):

- قراءة أسطر الجزء Hri من العلاقة r من القرص واحداً تلو الآخر.
- لكل سطر tr :

- نبحث عنه في فهرس الـ Hash الموجود في الذاكرة.
- في حال وجود أسطر مطابقة ts في Hsi :

- نخرج دمج (concatenation) السطرين tr و ts إلى الناتج

تسمى العلاقة r في هذه المرحلة بعلاقة السبر (الامتحان) $Probe\ Input$

اختيار عدد الأجزاء n Partitions: (أي أجزاء العلاقة / الجدول بعد تجزئتها باستخدام تابع التقطيع)

- يتم اختيار عدد الأجزاء n بحيث: كل جزء Hsi يجب أن يتسع في الذاكرة

ملاحظة:

أجزاء العلاقة (s) $build$ يجب أن تتسع في الذاكرة.
أجزاء العلاقة (r) $probe$ لا يشترط أن تتسع في الذاكرة.
يتم دائماً اختيار العلاقة الأصغر كـ $build\ input$ لتقليل الكلفة.

التجزئة العودية (Recursive Partitioning):

إذا كان: $n > M$

أي عدد الأجزاء أكبر من عدد صفحات الذاكرة، عندها:

- لا يمكن تحميل كل جزء في الذاكرة
- نلجأ إلى التجزئة العودية



كيف تتم؟

بدل التجزئة إلى n أجزاء:

- نقسم العلاقة s إلى $M - 1$ جزء
- نعيد تجزئة هذه الأجزاء باستخدام دالة Hash مختلفة
- نطبق نفس طريقة التجزئة على العلاقة r

كلفة خوارزمية Hash-Join:

يوجد لدينا حالتان:

1. عدم وجود تجزئة عودية (No Recursive Partitioning):

أي أن جميع الأجزاء الناتجة عن التجزئة الأولى يمكن تحميلها في الذاكرة.

- قراءة وكتابة العلاقتين أثناء مرحلة التجزئة: $2(br + bs)$
- مرحلة البناء والاختبار (Build & Probe) حيث نقرأ كل جزء من كل علاقة مرة واحدة: $(br + bs)$
- الكتل المملوءة جزئياً قد تؤدي إلى قراءة وكتابة كتلة إضافية لكل جزء: $2 \times nh$

الكلفة الكلية المتوقعة:

- $3(br + bs) + 2nh$

وغالباً يتم إهمال $2nh$ في المسائل التقريبية.

2. في حال تطلبت العملية تجزئة عودية (Recursive Partitioning):

أي أن بعض الأجزاء الناتجة عن التجزئة الأولى لا تتسع في الذاكرة.

- عدد مراحل التجزئة اللازمة لعلاقة Build (عادةً s) هو: $\lceil \log_{M-1}(bs) - 1 \rceil$
- عدد أجزاء العلاقة r هو نفسه عدد أجزاء العلاقة s ، لذلك يتم حساب عدد مراحل التجزئة لهما بالطريقة نفسها
- من الأفضل دائماً اختيار العلاقة الأصغر كـ build relation لتقليل عدد المراحل وبالتالي تخفيض الكلفة

الكلفة الكلية المتوقعة:

- $2(br + bs) \times \lceil \log_{M-1}(bs) - 1 \rceil + (br + bs)$

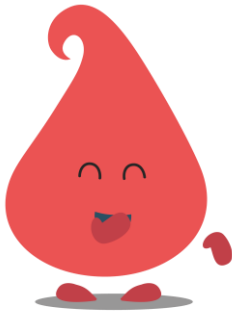
أفضل حالة (Best Case):

- إذا أمكن تحميل علاقة الـ build كاملة في الذاكرة الرئيسية، فلا نحتاج إلى تجزئة ولا إلى ملفات وسيطة، وتصبح الكلفة: $br + bs$

مثال تطبيقي:

لنفرض لدينا عملية دمج: customer $\bowtie$ depositor

- نستخدم Hash Join
- العلاقة depositor هي build input
- العلاقة customer هي probe input
- حجم الذاكرة المتاح = 20 كتلة



$$b(\text{customer})=400 \quad b(\text{depositor})=100$$

نختار:

$$nh=5$$

- أي خمس أجزاء، حجم كل جزء من depositor هو 20 كتلة ← تتم التجزئة بمرحلة واحدة فقط.
- يتم تجزئة customer أيضاً إلى خمس أجزاء، حجم كل جزء 80 كتلة ← أيضاً بمرحلة واحدة.
- بما أنه لا توجد تجزئة عودية، فإن الكلفة المتوقعة: $3(br + bs) = 3(400 + 100) = 1500$ كتلة منقولة

مع إهمال كلفة الكتل الإضافية الناتجة عن التعبئة الجزئية.

Complex Joins

1. Join: بشرط معقد (و) (Conjunctive Condition)

الشكل العام:

$$r \bowtie \theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n s$$

أي:

- الدمج بين r, s يتم فقط إذا تحققت جميع الشروط معاً

كيف ننفذه؟

لدينا طريقتان:

طريقة 1: Nested Loops / Block Nested Loops

- أثناء المقارنة بين كل زوج من الأسطر: نختبر كل الشروط $\theta_1, \theta_2, \dots$
- إذا تحققت جميعها نخرج الناتج
- هذه الطريقة مباشرة لكنها قد تكون مكلفة

طريقة 2:

- ننفذ دمجاً بسيطاً باستخدام شرط واحد فقط: $r \bowtie \theta_i s$
 - نحصل على نتيجة وسيطة
 - نطبق باقي الشروط عليها ك اختيار (Selection):
- $$\theta_1 \wedge \dots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \dots \wedge \theta_n$$

الفكرة هنا:

نفذ أولاً الشرط الأكثر انتقائية لتقليل حجم النتيجة.

2. Join: بشرط معقد (أو) (Disjunctive Condition)

الشكل العام:



$$r \bowtie \theta_1 \vee \theta_2 \vee \dots \vee \theta_n s$$

أي:

يكفي أن يتحقق أي شرط واحد فقط

كيف ننفذه؟

الطريقة 1: Nested Loops / Block Nested Loops

لكل زوج أسطر (واحد من r وواحد من s):

- نتحقق إن كان أحد الشروط θ_i صحيحاً
- إذا نعم نخرج الناتج

الطريقة 2: تفكيك OR إلى UNION

نحسب كل دمج على حدة ثم نأخذ الاتحاد:

$$(r \bowtie \theta_1 s) \cup (r \bowtie \theta_2 s) \cup \dots \cup (r \bowtie \theta_n s)$$

ملخص فكرة Complex Join:

الفكرة	نوع الشرط
كل الشروط يجب أن تتحقق	AND ($\wedge$)
يكفي تحقق شرط واحد	OR ($\vee$)
نقّذ Join واحد ثم Selection	AND Join
التتائج Joins منفصلة	OR Join
يصلح للحالتين	Nested Loops

Other Operations (عمليات أخرى في معالجة الاستعلامات)

1. إزالة التكرار (Duplicate Elimination)

تهدف هذه العملية إلى حذف السجلات المكررة والإبقاء على نسخة واحدة فقط.

الطرق المستخدمة:

1. باستخدام الفرز (Sorting):

- عند فرز السجلات:
- القيم المتطابقة تصبح متجاورة.
- نقوم بقراءة السجلات بالتسلسل:
- نحتفظ بأول سجل
- نحذف أي سجل يليه إذا كان مكرراً

2. باستخدام التقطيع (Hashing):

- نستخدم دالة تقطيع:
- كل القيم المتساوية تذهب إلى نفس الجزء (partition)
- داخل كل جزء:
- نقارن السجلات
- نحذف المكرر منها

2. الإسقاط (Projection)

- اختيار أعمدة محددة فقط من الجدول

كيف يُنفذ عملياً؟

1. نطبق الإسقاط على كل سجل (نحتفظ فقط بالأعمدة المطلوبة)
2. بعد ذلك نزيل التكرارات من العلاقة الناتجة

3. التجميع (Aggregation)

مثال:

- COUNT
- SUM
- AVG
- MIN
- MAX

مع أو بدون (GROUP BY)

طريقة التنفيذ:

- تنفذ بنفس فكرة إزالة التكرار تقريباً
- نستخدم: الفرز أو التقطيع (Hashing) لجمع السجلات التي تنتمي إلى نفس المجموعة (Group)

بعد التجميع:

- نطبق دوال التجميع على كل مجموعة بشكل مستقل

مثال:

بعد تجميع السجلات حسب department نحسب AVG (salary) لكل قسم.

4. عمليات المجموعات (Set Operations)

تشمل:

- Union (U)
- Intersection (∩)



Difference (-)

يمكن أن تستخدم عملية الدمج الممزوج بعد الفرز.

🔗 **تطبيق على Complex Joins:** لحساب عملية دمج تشمل ثلاث علاقات أو أكثر، مثل:

$$\text{loan} \bowtie \text{depositor} \bowtie \text{customer}$$
يمكن تنفيذ الدمج بعدة استراتيجيات، تختلف في **الكلفة والأداء**:

الطريقة 1: تنفيذ الدمج على مرحلتين (depositor $\bowtie$ customer)

1. نقوم أولاً بدمج: depositor $\bowtie$ customer

2. نستخدم الناتج الوسيط لدمجه مع: loan

أي:

$$\text{Loan} \bowtie (\text{depositor} \bowtie \text{customer})$$
ملاحظة:

- قد يكون الناتج الوسيط كبير الحجم
- مما يؤدي إلى كلفة قراءة وكتابة إضافية على القرص

الطريقة 2: تغيير ترتيب الدمج

1. نبدأ بدمج: loan $\bowtie$ depositor

2. ثم ندمج الناتج مع: customer

الفكرة:

تغيير ترتيب الدمج قد يؤدي إلى ناتج وسيط أصغر، وبالتالي كلفة أقل.

هذه الاستراتيجية تُستخدم عندما:

- يكون أحد الجداول أصغر
- أو عند وجود فهارس على واصفات الدمج

الطريقة 3: دمج العلاقات الثلاث دفعة واحدة (Multi-way Join)

يتم بناء فهرس على:

- العلاقة loan باستخدام loan-number
- العلاقة customer باستخدام customer-name

نقوم بالمرور على كل سجل من العلاقة: depositor

ولكل سجل:

- نبحث عن السجلات الموافقة له في customer
- ونبحث عن السجلات الموافقة له في loan

الميزة الأساسية:

- كل سجل من depositor يُفحص مرة واحدة فقط.
- لا يوجد ناتج وسيط كبير
- دمج عمليتين في عملية واحدة متخصصة

النتيجة:

هذه الاستراتيجية أكثر كفاءة من تنفيذ عمليتي دمج منفصلتين.

(Evaluation of Expressions) تقييم التعبير

يتم تقييم تعبيرات الاستعلام في نظم إدارة قواعد البيانات بطريقتين أساسيتين:

1. (التجسيد (Materialization):

في هذه الطريقة:

- يتم تنفيذ كل عملية موجودة في شجرة التعبير بشكل منفصل.
- بعد كل عملية، يتم تخزين النتيجة مؤقتًا على القرص على شكل علاقة بسيطة (Temporary Relation).
- تستخدم هذه العلاقة المخزنة كدخل للعملية التالية.

العيب الرئيسي:

- كثرة عمليات القراءة والكتابة على القرص.
- لذلك تعتبر هذه الطريقة مكلفة من حيث الزمن والكلفة.

2. (التنفيذ الأنبوبي (Pipelining):

في هذه الطريقة:

- يتم تنفيذ عدة عمليات في نفس الوقت.
- لا يتم تخزين ناتج العملية على القرص، بل يتم تمرير الأسطر الناتجة مباشرة إلى العملية التالية.
- أي أن الناتج ينتقل عبر أنبوب من عملية إلى أخرى.

المزايا:

- تقليل عمليات الوصول إلى القرص.
- كلفة أقل وزمن تنفيذ أسرع.
- لذلك يحاول نظام إدارة قواعد البيانات استخدام هذه الطريقة كلما أمكن.

لا تحزن على ما فات

احزن إن لم تتعلم منه

⚡ متى لا يمكن استخدام ال Pipelining ؟

بعض العمليات تمنع التنفيذ الأنبوبي وتجب النظام على استخدام التجسيد، مثل: Hash Join حيث يجب:

- تخزين العلاقتين كاملتين.
- تطبيق تابع التقطيع (Hash Function).
- تقسيم العلاقتين إلى أجزاء قبل البدء بالدمج.

لذلك لا يمكن تنفيذ Hash Join بطريقة أنبوبية، ويجب استخدام Materialization

شرط فعالية التنفيذ الأنبوبي:

لكي يكون التنفيذ الأنبوبي فعالاً:

- يجب استخدام خوارزميات تقييم **تنتج أسطر الخرج تدريجياً**
- أي أن طريقة توليد الخرج تكون مشابهة لطريقة استقبال الدخل (Tuple-at-a-time)

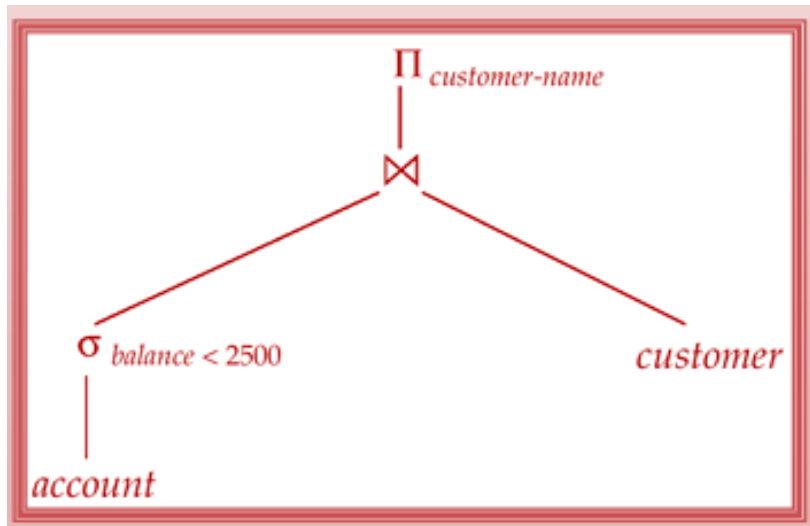
مثال توضيحي:

المطلوب:

إيجاد أسماء الزبائن الذين أرصدتهم أقل من 2500.

⚡ خطوات التنفيذ:

1. أخذ العلاقتين:
 - account
 - customer
2. تنفيذ عملية الدمج (Join)
3. تطبيق عملية الاختيار: $balance < 2500$
4. تنفيذ الإسقاط (Projection) على اسم الزبون



باستخدام Pipelining:

- لا يتم تخزين ناتج عملية الاختيار.
- يتم تمرير الأسطر التي تحقق الشرط مباشرة إلى عملية الدمج.
- وبالمثل، لا يتم تخزين ناتج الدمج، بل يُمرَّر مباشرة إلى عملية الإسقاط.

النتيجة لذلك:

- لا يوجد تخزين على القرص.
- كلفة أقل من طريقة Materialization.

The End...

**القوة الحقيقية أن تستمر
وأنت لا تملك إلا إيمانك بنفسك**

